

Aufgabe 1: Wörter aufräumen

Team-ID: 00048

Team: `while(!success){try();}`

Bearbeiter/-innen dieser Aufgabe:
Rui Zhang

21. November 2020

Inhaltsverzeichnis

Lösungsidee.....	1
Umsetzung.....	1
Beispiele.....	2
Quellcode.....	2

Lösungsidee

Die Lösungsidee der Aufgabe ist es, für jede Lücke in dem Lückentext alle möglichen einsetzbaren Wörter zu bestimmen. Danach werden die folgende Schritte wiederholt, bis alle Lücken ausgefüllt sind:

1. In allen Lücken, die noch nicht gelöst sind und nur ein Wort als mögliche Lösung besitzen, das Wort einsetzen.
2. Eingesetzte Wörter von allen übrigen Wörter löschen und die möglichen Lösungen der restlichen Wörtern aktualisieren, indem alle schon eingesetzten Wörtern von den Lösungen der restlichen Wörter gelöscht werden.

Da es in der Aufgabe angegeben wurde, dass es für jeden Lückentext nur eine richtige Lösung geben kann, kann es nicht zustande kommen, dass jedes Wort in einer Wiederholung des Algorithmus mehr als eine mögliche Lösung besitzt. Mindestens ein Wort muss nur eine mögliche Lösung besitzen. Schließlich gäbe es sonst mehrere Lösungen für den Lückentext.

Umsetzung

Das Programm zu der Aufgabe wurde in Java geschrieben. Es gibt zwei Klassen, die für das Lösen der Aufgabe relevant sind:

„LueckenRaetsel“: Diese Klasse liest eine Beispieldatei und erstellt daraus Lücken und speichert alle verfügbaren Wörter ab.

„Luecke“: Diese Klasse erzeugt ein Lücken-Objekt, das die Lücke selbst (als Text), alle einsetzbaren Wörter und die entgültige Lösung sowie weitere Attribute abspeichert.

Zunächst wird ein Objekt der Klasse „LueckenRaetsel“ erstellt, wobei der Pfad zur Beispieldatei angegeben wird. Eine Liste aus ungelösten Lückenobjekte und eine weitere aus Wörtern entstehen aus der Datei. Es wird anschließend die Methode „loeseRaetsel()“ aufgerufen, die die Lösungsidee umsetzt.

In dieser Methode wird ein Array von Lücken erstellt, das zunächst leer ist bzw. „Platzhalter Lückenobjekte“ enthält. Diese sind nichts anderes als Lücken ohne richtige Werte für die Attribute. Dieses Array wird am Ende alle gelösten Lückenobjekte enthalten.

Die Lösungsidee wird dann mithilfe einer while-Schleife umgesetzt, wobei nach jeder Iteration abgefragt wird, ob die Liste der ungelösten Lückenobjekte, die am Anfang bei der Instantiation des Lückenrätsels erstellt wurde, leer ist. Falls nicht, so wird durch die Liste von ungelösten Lückenobjekte durchgegangen. Eine If-Else-Abfrage bestimmt, ob die Anzahl der Lösungen des Objekts eins ist.

Falls ja, so wird die Lücke eingesetzt, indem das Lösungs-Attribut des Lückenobjekts aktualisiert wird. Die Lücke wird dann in das Array mit allen ausgefüllten Lücken an der richtigen Stelle eingesetzt, und das eingesetzte Wort wird von der Liste mit allen verfügbaren Wörtern gelöscht. Das Lückenobjekt selber wird von der Liste aller ungelösten Lückenobjekte gelöscht, da dieses jetzt gelöst wurde.

Wenn nein, dann werden alle Wörter in der Liste aller möglichen Lösungen dieses Lückenobjekts entfernt, die nicht in der Liste aller verfügbaren Wörtern des Lückentexts vorhanden ist.

Zur Bestimmung der einsetzbaren Wörtern in einer Lücke wird eine Methode in der „Luecken“ Klasse angewendet. Die Methode vergleicht die Darstellung der Lücke des Lückenobjekts in Zeichenkettenform mit allen Wörtern, die dieselbe Zeichenkettenlänge besitzen wie die Lücke, indem durch jedes Zeichen durchgegangen wird, und abgefragt wird, ob das Zeichen der Lücke an dieser Stelle mit dem Zeichen des Wortes an derselben Stelle gleich sind, oder ob die Lücke an dieser Stelle einen freien Platz besitzt („_“), wo ein beliebiges Zeichen eingesetzt werden kann. Wenn keines der beiden Fälle zutrifft, dann wird dieses Wort nicht zu der Liste aller möglichen Lösungen für das Lückenobjekt hinzugefügt, falls alle Zeichen jedoch passen, dann wird das Wort hinzugefügt.

Beispiele

Die Beispiele aus der BWINF Webseite wurden für das Testen des Programms genutzt.

In jeder Beispieldatei war mindestens eine Lücke vorhanden, welche ein Satzzeichen am Ende besaß. Auf Satzzeichen musste das Programm also achten. Dieses Problem wurde mithilfe eines Attributs in der „Luecken“ Klasse namens „satzzeichen“ gelöst.

Quellcode

```
public class LueckenRaetsel {  
    private final ArrayList<String> woerter;  
    private final ArrayList<Luecke> luecken;  
  
    public LueckenRaetsel(String path) throws IOException, NoSuchElementException {  
        BufferedReader reader = new BufferedReader(new  
InputStreamReader(getClass().getResourceAsStream(path), StandardCharsets.UTF_8));  
  
        String[] lueckenText = reader.readLine().trim().split(" ");  
        this.woerter = new ArrayList<>(Arrays.asList(reader.readLine().split(" ")));  
        this.luecken = new ArrayList<>();  
  
        for (int i = 0; i < lueckenText.length; i++) {  
            this.luecken.add(new Luecke(lueckenText[i], this.woerter, i));  
        }  
    }  
  
    public String loeseRaetsel() {  
        Luecke[] ausgefuellteLuecken = new Luecke[this.luecken.size()];  
        for (int i = 0; i < ausgefuellteLuecken.length; i++) {  
            ausgefuellteLuecken[i] = new Luecke();  
        }  
  
        while (!this.luecken.isEmpty()) {  
  
            for (int i = 0; i < this.luecken.size(); i++) {  
                Luecke lueckenObjekt = this.luecken.get(i);  
                if (lueckenObjekt.getMoeglicheLoesungen().size() == 1) {
```

```
        ausgefuellteLuecken[lueckenObjekt.getLueckenIndex()] = lueckenObjekt;
        lueckenObjekt.setLoesung(lueckenObjekt.getMoeglicheLoesungen().iterator().next());
        this.luecken.remove(lueckenObjekt);
        this.woerter.remove(lueckenObjekt.getLoesung());
        i--;
    } else {
        lueckenObjekt.getMoeglicheLoesungen().removeIf(loesung -> !
this.woerter.contains(loesung));
    }
}

}

StringBuilder outputString = new StringBuilder();
for (Luecke luecke : ausgefuellteLuecken) {
    if (luecke.getSatzZeichen() != Luecke.keinSatzZeichen)
        outputString.append(luecke.getLoesung()).append(luecke.getSatzZeichen()).append(" ");
    else
        outputString.append(luecke.getLoesung()).append(" ");
}

return outputString.toString();
}
}
```

```
public class Luecke {
    public static final LinkedList<Character> alleSatzZeichen;
    public static final char keinSatzZeichen;
    static {
```

```
alleSatzZeichen = new LinkedList<Character>(){  
    add('!');  
    add(',');  
    add('.');  
    add(':');  
    add('?');  
};  
keinSatzZeichen = '-';  
}
```

```
private final String luecke;  
private final String[] alleWoerter;  
private final int lueckenIndex;  
private char satzZeichen;  
private String loesung;  
private final HashSet<String> moeglicheLoesungen;
```

```
public Luecke() {  
    this.luecke = null;  
    this.alleWoerter = new String[] { };  
    this.lueckenIndex = -1;  
    this.satzZeichen = keinSatzZeichen;  
    this.loesung = null;  
    this.moeglicheLoesungen = null;  
}
```

```
public Luecke(String luecke, ArrayList<String> alleWoerter, int lueckenIndex) {  
    this.luecke = luecke;  
    this.alleWoerter = new String[alleWoerter.size()];  
}
```

```
for (int i=0; i<alleWoerter.size(); i++) {
    this.alleWoerter[i] = alleWoerter.get(i);
}

this.lueckenIndex = lueckenIndex;
this.satzZeichen = keinSatzZeichen;
this.loesung = null;
this.moeglicheLoesungen = new HashSet<>(this.findeLoesungen());
}

public int getLueckenIndex() { return this.lueckenIndex; }
public char getSatzZeichen() { return this.satzZeichen; }
public String getLoesung() { return this.loesung; }
public HashSet<String> getMoeglicheLoesungen() { return this.moeglicheLoesungen; }

public void setLoesung(String newLoesung) { this.loesung = newLoesung; }

private LinkedList<String> findeLoesungen() {
    LinkedList<String> passendeWoerter = new LinkedList<>();

    String lLuecke = this.luecke;
    char lastChar = lLuecke.charAt(lLuecke.length()-1);

    if (alleSatzZeichen.contains(lastChar)) {
        this.satzZeichen = lastChar;
        lLuecke = lLuecke.substring(0, lLuecke.length()-1);
    }
}
```

```
    for (String wort : this.alleWoerter) {  
        if (wort.length() == lLuecke.length() && this.lueckenWortCheck(lLuecke, wort)) {  
            passendeWoerter.add(wort);  
        }  
    }  
    return passendeWoerter;  
}
```

```
private boolean lueckenWortCheck(String pLuecke, String wort) {  
    for (int i=0; i<pLuecke.length(); i++) {  
        if (wort.charAt(i) != pLuecke.charAt(i) && pLuecke.charAt(i) != '_') {  
            return false;  
        }  
    }  
    return true;  
}  
}
```